University of Aberdeen

School of Natural and Computing Sciences

Department of Computing Science

2024 – 2025

Programming assignment – Individually Assessed (no teamwork)

Title: JC2503 – Web Application Development	Note: This assignment accounts for 30% of your total mark of the course.

Learning Outcomes

On successful completion of this component a student will have demonstrated to be able to:

- Write and run basic Web applications
- Judge how websites and web applications should be structured
- Apply techniques to support producing correct JavaScript code in Node.js platform
- Use Socket.io to implement interactive elements in web applications

Information for Plagiarism and Collusion: The source code and your report may be submitted for plagiarism check in *MyAberdeen*. Please refer to the slides available at *MyAberdeen* for more information about avoiding plagiarism before you start working on the assessment. Excessive use of generative AI (GenAI) tools, such as ChatGPT or DeepSeek, for writing the code or the report can also be considered as plagiarism. You can use GenAI for proofreading or assisting to write small pieces of code, but the general structure of your code and report should be developed by yourself. As this is individual work, submitting similar work with another student can be considered as collusion. Also read the following information provided by the university:

https://www.abdn.ac.uk/sls/online-resources/avoiding-plagiarism/

Extension Policy: According to the extension policy of University of Aberdeen, teachers are not allowed to grant deadline extensions for coursework assignments. Extensions may be requested from the school administration by e-mail: uoa-ji-enquiries@abdn.ac.uk.

Extensions require strong justifications (such as serious illness or mourning), and extension requests should be accompanied with supporting evidence, such as a medical certificate. For unauthorised late submissions, 8 points of the mark will be deduced for the first 24 hours, and then 4 points for each consecutive day, up to 7 days. Submissions more than 7 days late will receive zero marks.

1. Introduction

In this assignment, your task is to build a simple website containing information about yourself, as well as an interactive puzzle game application for multiple players. The website should contain three pages. The first one (root) is the introduction page where you tell briefly who you are, what you study, etc. The second page will contain more detailed information about you from the chosen point of view. You can, for example, explain your professional interests, what courses have you taken, and what topics in your field you are especially interested in. Alternatively, you could also explain something about your leisure activities and achievements for example in some hobby that is important for you. The third page should contain a game application, with more detailed requirements explained later in this document.

The webpages should include a navigation system allowing user to move easily from each page to another. The first two pages can be static pages implemented in HTML and CSS, but for the third page, you need to implement also back-end functionality using JavaScript on Node.js platform. The final project, including HTML, CSS, and JavaScript code and the project report, should be submitted in *MyAberdeen*. The high-level page structure is shown in the picture below.

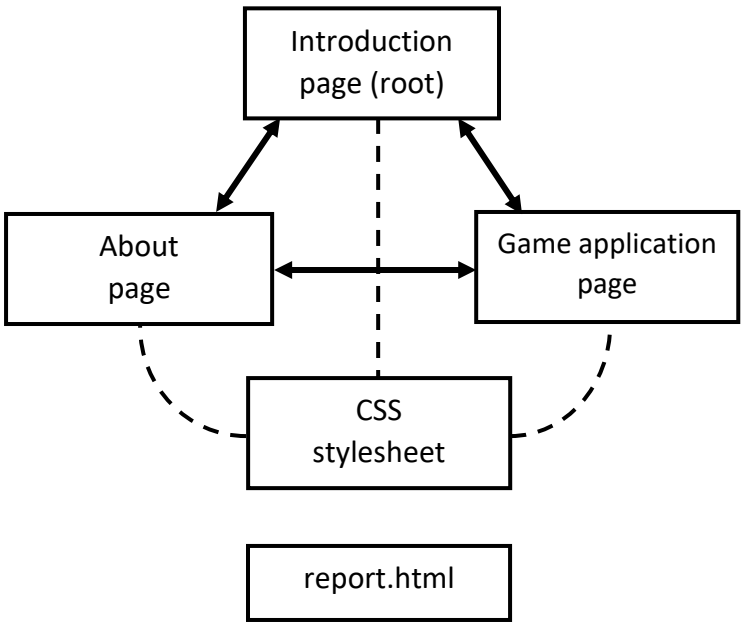

Figure 1. Illustration of the website structure.

2. General Guidance and Requirements

Your assignment code and report must conform to the requirements given below and include the required content as outlined in each section. You **must** supply a written report, along with the corresponding code, containing all distinct sections/subtasks that provide a full critical and reflective account of the processes undertaken.

While we are not marking your use of Git in this assignment, you are strongly encouraged to do this work inside a Git repository. By doing that you gain the benefit of rolling back your edits, or

doing work in branches, and then merging it to the main branch as you accomplish tasks along the way.

To facilitate testing and marking, we will test all implementations using a Node.js environment with a fixed configuration. You should install Node.js following the instructions in Practical 1. We recommend installing Node.js version 24.13.0 (LTS), as we use this version for testing. You should make a folder for your assignment, download ***package.json*** file available on MyAberdeen into that folder, and then run **`npm install`** command to configure the environment. This will install dependencies including Express, Socket.io, and EJS, that are essential to complete the assignment. You should name your main JavaScript file as **server.js** so that it will start when you use command **`npm start`**.

After you have finished your project and it is ready for submission, you should pack your project files and subdirectories like **public** and their contents into a single compressed file (e.g., **.zip** or **.rar**). However, *do not* include **package.json** file and **node_modules** directory, since these are supposed to be identical in all projects.

Do not change the *package.json* file provided on MyAberdeen. We cannot run **`npm install`** separately for every student. With almost 300 students attending the course, even a quick two-minute installation per submission would add up to around **10 hours** of processing time. Because of this, **your server must run using the default environment**. No extra packages can be installed when we test the submissions.

If your server does not run in the test environment because you modified **package.json** or your server relies on additional dependencies, we will only be able to mark your **static webpages**, and you will receive **zero marks** for the application (server) part. If you want to test that your submission complies with the requirements, you can create a new empty folder (you can use different computer, if you have access to one), download **package.json** from MyAberdeen in there, run **`npm install`**, extract your compressed file to that folder, and try to run the server using command **`npm start`**. If your server still works properly, your implementation is valid.

2.1 Requirements and Marking Criteria for the Website Structure [20 marks]

The Introduction page (root page) should include a personal introduction and an overview of the page in general. The About page should include more detailed information about an aspect of yourself you choose, for example your professional or private interests. You can decide the content for the Introduction and About pages yourself. The third page will contain game application with more specific requirements given in a separate section, but the HTML code for the game application should also follow the criteria below. There should be enough content to demonstrate that you can create HTML pages with different elements, such as header, footer, paragraphs, images, sidebar, etc.

The marking criteria for the web structure are the following:

- The HTML code is written in valid HTML 5 and passes HTML validation **[10 marks]**. For any validation errors, 2 marks are deducted, and for any warnings, 1 mark is deducted. Marks can also be deducted if the code is very short or trivial. Minimum mark is 0.
- There is a functional navigation system between the pages **[5 marks]**.
- The content is appropriate and organised logically **[5 marks]**.

2.2 Requirements and Marking Criteria for the Styling and Design [20 marks]

Your webpages must be styled with CSS to improve the presentation of the content and aid the user in using the game application. If you use Bootstrap, you must include a custom style sheet with valid CSS modifying the default Bootstrap styling.

You should include enough CSS code to demonstrate that you can use CSS to define different colours, layouts, etc. You can choose the colours and style of the website yourself, but you should follow the chosen style consistently on all the three pages. While creativity is appreciated, usability is more important than beautiful design: you should avoid using large videos or flashy animations that can be distractive and slow to download.

The marking criteria for the style and design are the following:

- CSS code passes CSS validation **[10 marks]**. For any validation errors, 2 marks are deducted, and for any warnings, 1 mark is deducted. Marks can also be deducted if the code is very short or trivial. Minimum mark is 0.
- Visual appearance, responsiveness, accessibility, consistency of the design **[10 marks]**.

2.3 Requirements and Marking Criteria for the Game Application [30 marks]

The application shall allow users to connect through a web browser, register with a chosen display name, and participate in a continuously running turn-based puzzle game involving shaped and coloured blocks placed on a shared 4×4 grid.

When accessing the game page, a user shall be prompted to enter a player name. Once the name has been submitted, the user shall be added to the active player list and placed at the end of the current turn order. The system shall permit new players to join at any time while the game is in progress. A player may leave at any point; if they disconnect or fail to act on their turn within a designated timeout interval (e.g., one minute), the server shall treat this as player departure and remove them from the game.

The game board shall consist of a 4×4 grid containing 16 positions. The block set shall contain exactly 16 unique blocks, produced by combining four shapes with four colours. Any choice of four shapes and four colours is acceptable provided each combination produces one unique block; an example is shown in Figure 2. At the start of the game, the grid shall be empty, and all blocks shall be available in a shared pool.

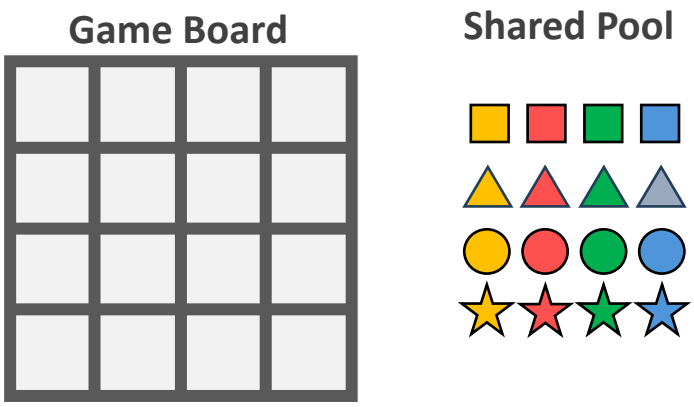

Figure 2. Puzzle game board and example blocks.

The game shall proceed in strict turn order. When it is a player's turn, the server shall select one block uniformly at random from the pool of available blocks and present it exclusively to that player. The player shall select one empty position on the grid by clicking that position. Upon selection, the block shall be placed in the chosen position and removed from the available pool. The turn ends immediately after placement. An example of one round of the game is shown in Figure 3.

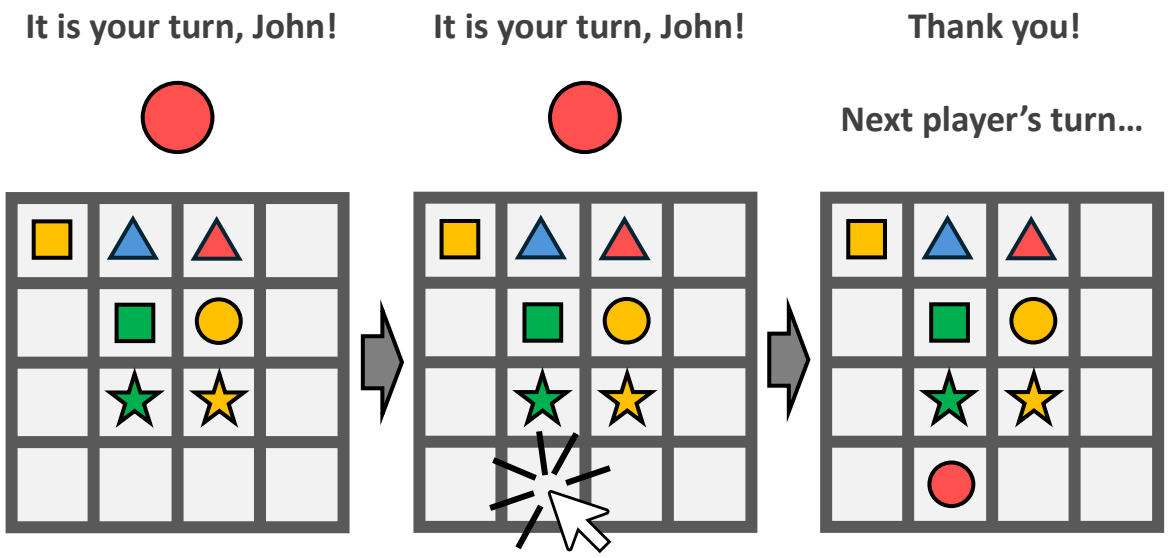

Figure 3. Example play of one round of the game.

Immediately after a block is placed, the system shall check whether this placement results in a contiguous line of three or more blocks that share either the same shape or the same colour. Lines may occur horizontally, vertically, or diagonally. If such a line exists, the system shall remove the matching blocks from the board, return them to the available pool, and award the player one point per removed block. An example is shown in Figure 4.

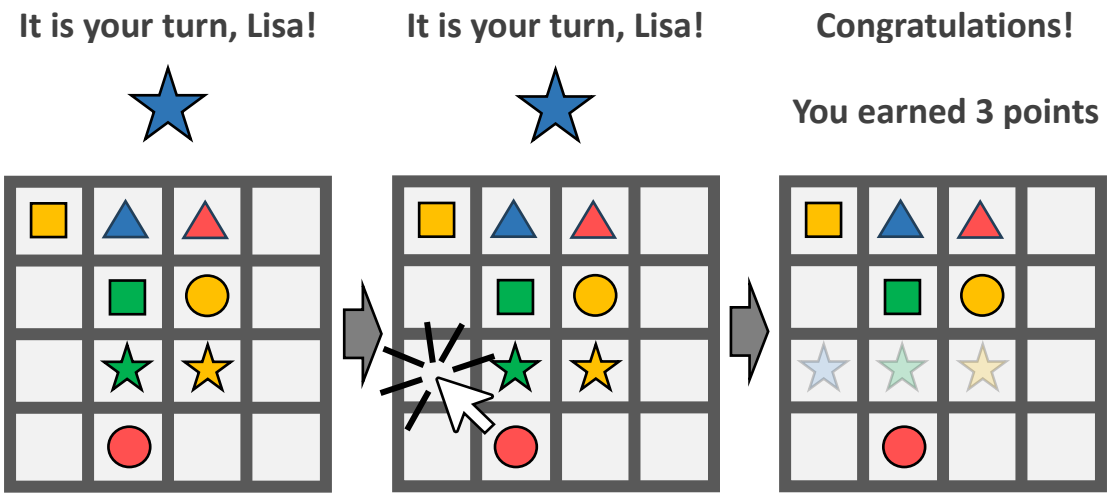

Figure 4. Example of clearing three blocks in a row.

If, after a placement, the board contains no empty positions (i.e., all 16 grid cells are occupied), the system shall clear the entire grid and return all blocks to the pool. The player whose turn produced the full grid shall receive a jackpot worth 16 points.

The game shall run continuously without terminating. Turn order shall advance cyclically through the active player list, adjusting dynamically as players join or leave. When a new player joins, their position will be the last in the list of players waiting for their turn. The system shall maintain a persistent game state, including the contents of the grid, the pool of available blocks, the player order, and the scoreboard.

Different functionalities of the game are marked as follows:

- The basic game logic is functional as specified above: blocks are added and removed from the board correctly **[10 marks]**.
- The server manages multiple players as specified above: new players can join the game, each player is given their turn, scoreboard works as required, etc. **[10 marks]**.
- General usability of the game application: at each stage, players are properly informed about the status of the game, the user interface is logical and responsive, etc. **[10 marks]**.

Partial marks can be awarded if a feature is implemented, but it does not work correctly due to a programming error. Note that the style of the game application will be assessed as part of the styling and design for the whole website.

2.4 Requirements and Marking Criteria for the Project Report [30 marks]

You should write the report in HTML format and save it as **report.html** among the other HTML files. You should consider the report as a standalone webpage: you can use plain HTML to format the report, and you do not need to include a link to the report in the navigation system. However, the report page should open with URL **localhost:8080/report.html** when the server is running. If you wish, you can use internal or inline CSS to format the report, but it is not required and does not give you any additional marks.

Your report should describe the overall design of the website and the game application, including how the game application client communicates with the server, with respect to handling events and using Socket.IO. The report should also include a section for reflection, detailing challenges faced, and lessons learned during the work. Appropriate references should be included, and if you have used GenAI tools at any stage, this should be acknowledged. The recommended length of the report is 600-800 words. Technical illustrations are encouraged, but you should not include screenshots.

The marking criteria for the report are the following:

- Structure and completeness (all the required aspects are covered) **[10 marks]**.
- Clarity and readability (language and illustrations are understandable) **[10 marks]**.
- The report is consistent with the implementation **[5 marks]**.
- References are adequate **[5 marks]**.

3. Submission

You should submit your full assignment, including all the HTML, CSS, JavaScript, and image files in compressed file format in MyAberdeen. **The deadline is May 26 at 23:59 (Chinese time).**

4. Contact

For any questions or clarifications, you can contact the course teachers: Dr Xiaonan Liu (xiaonan.liu@abdn.ac.uk), Dr Yuan Wen (yuan.wen@abdn.ac.uk), and Dr Jari Korhonen (jari.korhonen@abdn.ac.uk).